计算机领域本科教育教学改革试点
工作计划（“101计划”）研究成果

数据结构

授课教师：谭光华

湖南大学　信息科学与工程学院

第 2 章

线性表

提纲

2.1　问题引入：一元多项式
2.2　线性表的定义与结构
2.3　线性表的顺序存储实现
2.4　线性表的链式存储实现
2.5　线性表的应用
2.6　拓展延伸
2.7　应用场景：内存管理
2.8　小结

2.1 问题引入：一元多项式

- **例：一元多项式及其运算**
  - **一元多项式：** $f(x) = a_0 + a_1 x + \cdots + a_{n-1} x^{n-1} + a_n x^n$
  - **主要运算：多项式相加、相减、相乘等**

- **问题**
  - **如何在计算机中表示一元多项式，并实现相关运算**
  - **多项式的关键数据：多项式项数，每一项系数（以及对应指数）**

2.1 问题引入：一元多项式

- **方法1：采用顺序存储结构直接表示一元多项式**
  - **用数组a存储多项式的相关数据，数组分量a[i]表示项x^i的系数**
  - **例如：** $4x^5-3x^2+1$

$a[i]$	1	0	-3	0	0	4	……
下标i	0	1	2	3	4	5	……

- **优点**
  - **多项式相加容易**
- **缺点**
  - **如何表示** $1+2x^{30000}$

2.1问题引入：一元多项式

- **方法2：采用顺序存储结构表示多项式的非零项**
  - **每个非零项 $a_i x^i$ 涉及两个信息：指数i和系数a_i**
  - **可以将一个多项式看成是一个(a_i，i)二元组的集合**

$$\{(a_n, n), (a_{n-1}, n-1), \ldots\ldots, (a_0, 0)\}$$

$$P_1(x) = 9x^{12} + 15x^8 + 3x^2$$

系数	9	15	3	-
指数	12	8	2	-
数组下标i	0	1	2	……

$$P_2(x) = 26x^{19} - 4x^8 - 13x^6 + 82$$

系数	26	- 4	- 13	82	-
指数	19	8	6	0	-
数组下标i	0	1	2	3	……

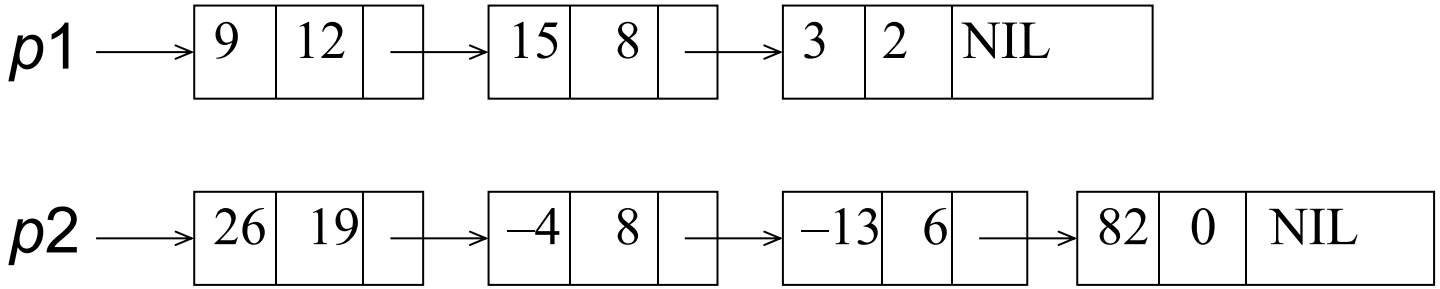
- **这种表示的优点和缺点?**

2.1问题引入：一元多项式

- **方法3：采用链表结构来存储多项式的非零项**
  - **链表表示多项式：每个链表结点对应一个非零项，包括**
    - **系数、指数、指针域**

系数	指数	指针

$$P_1(x) = 9x^{12} + 15x^8 + 3x^2 \qquad P_2(x) = 26x^{19} - 4x^8 - 13x^6 + 82$$



- **这种表示的优点和缺点?**

2.1问题引入：一元多项式

- **启示：数据结构的设计**
  - **往往需要在算法简洁、可理解性与时间、空间效率之间权衡**
  - **针对具体问题选择合适的数据结构及设计相应的算法**
- **数组和链表均是一种线性数据结构，称为线性表**
  - **数组和链表只是线性表的两种不同物理实现**

2.2.1 线性表的定义

- **线性表：由同一类型的数据元素构成的有序序列的线性结构**

代码2-1：线性表的抽象数据类型定义

ADT List {

数据对象：

{ a_i| a_i∈ElemSet，i=1,2,…n, n>0 } 或Φ，即空表；ElemSet为元素集合。

数据关系：

{<a_i, a_{i+1}>|a_i, a_{i+1}∈ElemSet，i=1,2,…n-1 }，a_1为表头元素，a_n为表尾元素。

基本操作：

InitList(*list*)：初始化一个空的线性表*list*。

DestroyList(*list*)：释放线性表*list*占用的所有空间。

Clear (*list*)：清空线性表*list*。

IsEmpty (*list*)：当线性表*list*为空时返回真值，否则返回假值。

Length (*list*)：返回线性表*list*中的元素个数，即表的长度。

Get (*list, i*)：返回线性表*list*中第*i*个元素的值。

Search(*list, x*)：在线性表*list*中查找元素*x*，查找成功，返回*x*的位置，否则返回NIL。

Insert (*list, i, x*)：在线性表*list*的第*i*个位置上插入元素*x*。

Remove (*list, i*)：从线性表*list*中删除第*i*个元素。

}

2.2.2 线性表的结构

- **线性表的逻辑结构**
  - 数据元素之间**线性的序列**关系，即数据元素之间的前驱和后继关系。
- **线性表的物理结构**
  - 线性表在计算机中的存储方式，又称为**存储结构**，即从程序实现的角度将逻辑结构映射到计算机存储单元中。
    - **顺序存储结构（顺序表）**：数据元素被顺序地存储在连续的内存空间中，前驱和后继元素在物理空间上是相邻的。
    - **链式存储结构（链表）**：前驱和后继元素在物理空间上不相邻，并使用类似指针这样的手段将结点按顺序前后链接起来

2.3 线性表的顺序存储实现

- **顺序表**
  - **基于数组实现**
  - **例如：数组list.data[kMaxSize]:**
    - **从list.data[0]开始依次顺序存放，kMaxSize代表数组最大容量，**
    - **list.last记录当前线性表中最后一个元素在数组中的位置，表空时list.last= -1。**

last

data	a_1	a_2	……	a_i	a_{i+1}	……	a_n	……	-
i	0	1	……	*i*-1	*i*	……	*n*-1	……	kMaxSize-1

  - **主要操作：初始化、查找、插入、删除**

2.3.1 顺序表的基本操作

- **1. 初始化：即构造一个空表**
  - **（1）动态分配表结构所需的存储空间**
  - **（2）将表中list.last指针置为-1，表示表中没有数据元素。**

代码 2-2 产生一个初始空顺序表 InitList(*list*)

输入： 顺序表 *list*

输出： 完成了初始化的空顺序表 *list*

```
list.data ← new ElemSet[kMaxSize]    //申请顺序表空间
list.last ← -1;                      //空表中 list.last 值为-1
```

2.3.1 顺序表的基本操作

- **2. 查找：在线性表中查找与给定值x相等的元素**
  - **从第一个元素 起依次和x比较，直到找到一个与x相等的数据元素，返回它在顺序表中的存储下标；查遍整个表都没有找到与x相等的元素，则返回NIL。**

算法 2-1 在顺序表 *list* 中查找元素 *x*　Search(*list*, *x*)

输入： 顺序表 *list*，$x \in$ ElemSet

输出： 元素 *x* 在顺序表 *list* 中的位置 *i*（由于顺序表中的位置从 0 开始，*x* 的实际位序是 *i*+1）；
如果 *x* 不在顺序表中返回 NIL

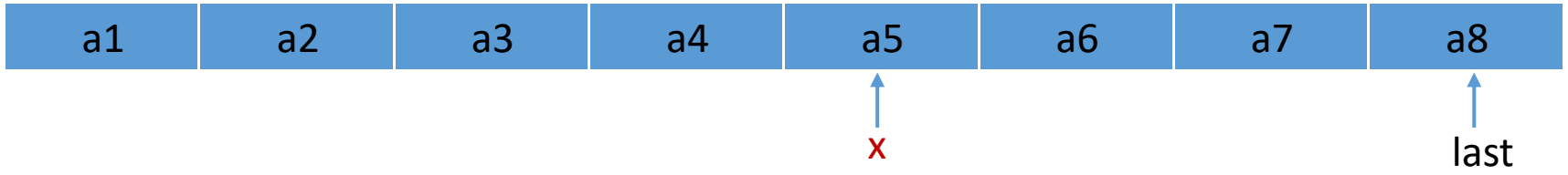
```
i ← 0;
while i ≤ list.last 并且 list.data[i] ≠ x do
|   i←i+1
end
if  i > list.last then     // 如果没找到，返回 NIL
      i ← NIL
end
return i
```

时间复杂性：

- 比较次数：1~n
- 平均：O(n)

2.3.1 顺序表的基本操作

- **3. 插入**
  - **在表的第i个位序上插入一个值为x的新元素**
  - $(a_1, a_2, \ldots, a_{i-1}, a_i, a_{i+1}, \ldots, a_n)$ --> $(a_1, a_2, \ldots, a_{i-1}, x, a_i, a_{i+1}, \ldots, a_n)$
  - **执行步骤：**
    - （1）将$a_i \sim a_n$顺序向后移动（**移动次序从后到前**），为新元素让出位置；
    - （2）将x置入空出的第i个位序；
    - （3）修改 list.last 指针(相当于修改表长)，使之仍指向最后一个元素。



2.3.1 顺序表的基本操作

- **3. 插入**

算法 2-2 在顺序表 *list* 的第 *i* 个位置上插入元素 *x*　Insert (*list, i, x*)

输入： 顺序表 *list*，*i* 是插入位置的序号（从 1 开始），$x \in$ ElemSet

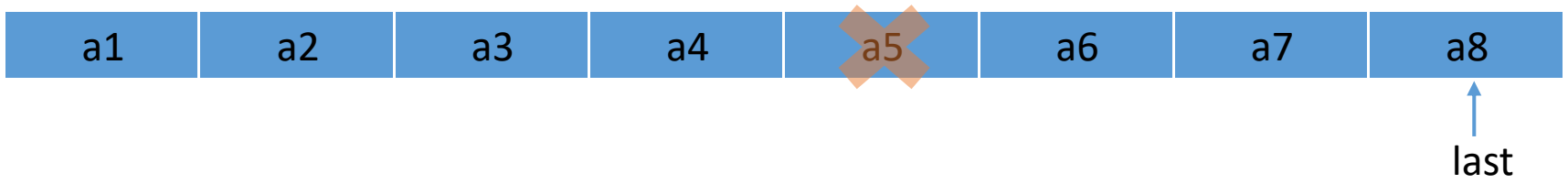
输出： 完成插入后的顺序表 *list*

```
if  list.last = kMaxSize－1 then    // 表空间已满，不能插入
|   表满不能插入，退出
end
if  i < 1 或者 i > list.last + 2 then   //检查 i 的合法性。注意 i 代表位序，不是数组下标
|   插入位置不合法，退出
end
for  j ← list.last downto i-1 do
|   list.data[j+1] ← list.data[j]   //将 a_i~a_n 顺序向后移动
end
list.data[i-1] ← x   // 新元素插入
list.last ← list.last + 1 // list.last 仍指向最后元素
```

- **时间复杂度**
  - **平均O(n)**

2.3.1 顺序表的基本操作

- **4. 删除**
  - **将线性表的第i个位序上的元素删除**
  - $(a_1, a_2, \ldots, a_{i-1}, a_i, a_{i+1}, \ldots, a_n)$ --> $(a_1, a_2, \ldots, a_{i-1}, a_{i+1}, \ldots, a_n)$
  - **执行步骤：**
    - (1)将$[a_{i+1}, a_n]$顺序向前移动，元素a_i被覆盖；
    - (2)修改list.last指针(相当于修改表长)使之仍指向最后一个元素。



2.3.1 顺序表的基本操作

- **4. 删除**

算法 2-3 从顺序表 *list* 中删除第 *i* 个元素　Remove (*list*, *i*)

输入： 顺序表 *list*，*i* 是删除元素的位置序号（从 1 开始）

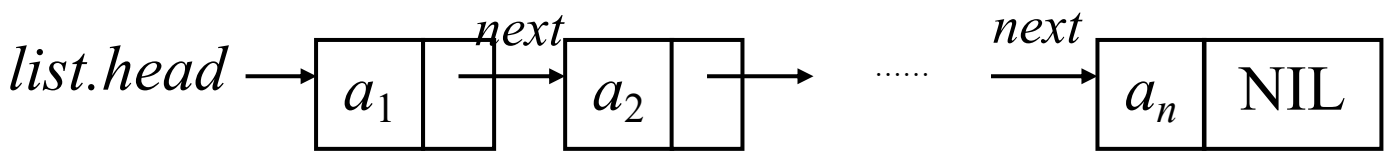
输出： 完成删除后的顺序表 *list*

```
if  i<1 或 i > list.last + 1 then   // 检查空表及删除位置的合法性
|   不存在这个元素，退出
end
for  j←i to list.last do
|   list.data[j-1]← list.data[j]   // 将 a_{i+1} ~ a_n 顺序向前移动
end
list.last ←list.last – 1   // list.last 仍指向最后元素
```

- **时间复杂度**
  - **平均O(n)**

2.4 线性表的链接存储实现

- **单向链表（简称单链表）**
  - **每个数据单元由数据域data和链接域next两部分组成，n个数据单元通过链接域next串联起来。**
  - **head指向第一个结点**



  - **不要求逻辑上相邻的两个数据元素物理上也相邻**
  - **通过“链”建立起数据元素之间的逻辑关系**
  - **对线性表的插入、删除不需要移动数据元素，只需要修改“链”**

2.4.2 单链表的基本操作

- **1. 求表长：单链表元素个数**
  - **（1）设一个移动指针p和计数器counter，初始化**
  - **（2）p逐步往后移，同时计数器counter加1**
  - **（3）当后面不再有结点时，counter的值就是结点个数，即表长。**

算法 2-4 求单链表 *list* 中的元素个数，即表长 Length (*list*)

输入： 单链表 *list*，其中 *list.head* 指向链表头结点

输出： 单链表长度

```
p ← list.head
counter ← 0
while  p ≠ NIL  do
|   counter ← counter + 1
|   p ← p.next
end
return counter
```

时间复杂度O(n)

2.4.2 单链表的基本操作

- **2. 查找**
  - （1）从链表的第一个元素结点起，判断当前结点其值是否等于x
  - （2）若是，返回该结点的位置（即指向该结点的指针），否则继续后一个，直到表结束为止
  - （3）找不到时返回空（NIL）

算法 2-6 在单链表 *list* 中查找元素 *x* 所在结点　Search(*list*, *x*)

输入： 单链表 *list*，其中 *list.head* 指向链表头结点；$x \in$ ElemSet

输出： 元素 *x* 在单链表 *list* 中的位置，即指向该结点的指针；如果 *x* 不在单链表中返回 NIL

```
p ← list.head
while  p ≠ NIL 且 p.data ≠ x  do
    p ← p.next
end
return p
```

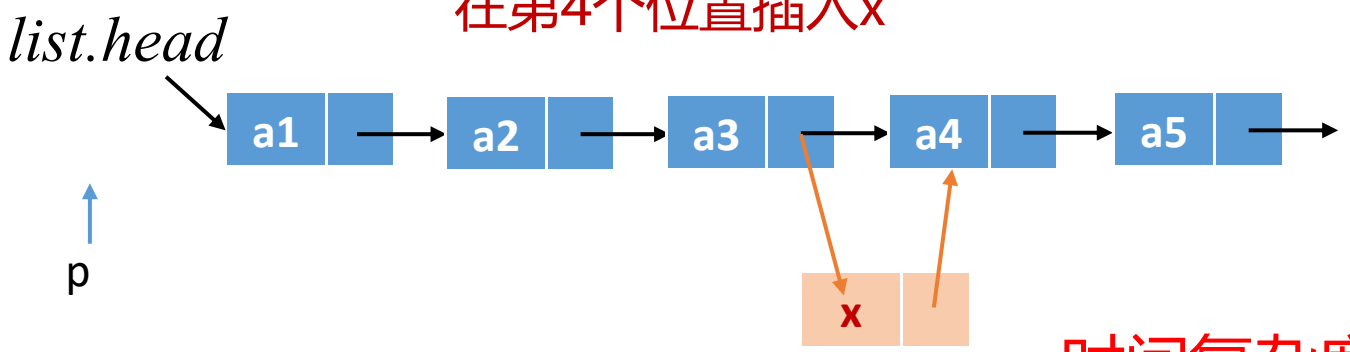
时间复杂度O(n)

2.4.2 单链表的基本操作

- **3. 插入：在list的第i个位置上插入元素x**
  - （1）找到第i-1个结点；
  - （2）若存在，则申请一个新结点的空间并填上相应值x，然后将新结点插到第i-1个结点之后；
  - （3）如果不存在则直接退出

插入表头和插入其他位置处理不同

在第4个位置插入x



时间复杂度O(n)

算法 2-7 在单链表 *list* 的第 *i* 个位置上插入元素 *x* Insert(*list*, *i*, *x*)

输入： 单链表 *list*，*i* 是插入位置的序号（从 1 开始），$x \in$ ElemSet

输出： 完成插入后的单链表 *list*

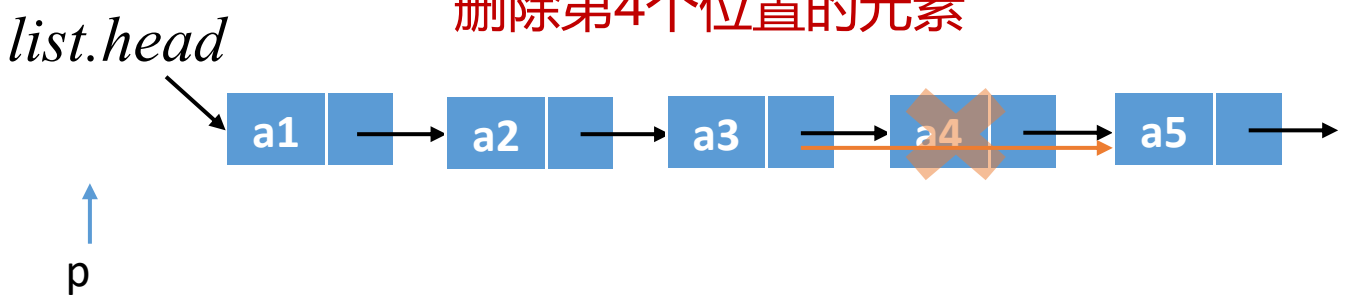
```
if i < 1 then
    插入位置不合法，退出
end
if i = 1 then    //插入第 1 个结点
    new_node ← new ListNode //创建新的结点
    new_node.data ← x
    new_node.next ← list.head //插入表头
    list.head ← new_node
else    // i > 1，寻找第 i-1 个结点并插入其后
    p ← list.head
    counter ← 1
    while p ≠ NIL 且 counter < (i-1) do
        p ← p.next
        counter ← counter + 1
    end
    if p ≠ NIL then //p 指向第 (i-1)个结点
        new_node ← new ListNode //创建新的结点
        new_node.data ← x
        new_node.next ← p.next
        p.next ← new_node
    else
        插入位置不合法，退出
    end
end
```

2.4.2 单链表的基本操作

- **4. 删除：删除单链表中指定位序i的元素**
  - **（1）找到被删除结点的前一个元素；**
  - **（2）再删除结点并释放空间**

删除表头和删除其他位置处理不同

删除第4个位置的元素



时间复杂度O(n)

算法 2-8 从单链表 *list* 中删除第 *i* 个元素 Remove (*list*, *i*)

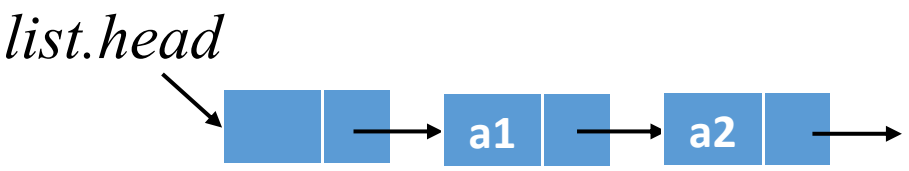
输入： 单链表 *list*，*i* 是删除元素的位置序号（从 1 开始）

输出： 完成删除后的单链表 *list*

```
if  i < 1  then
|  删除位置不合法，退出
end
p ← list.head
if  p ≠ NIL 且 i = 1  then  //删除第 1 个结点
|  list.head ← p.next
|  delete  p
else  // i > 1，寻找第 i-1 个结点
|  counter ← 1
|  while  p ≠ NIL 且 counter < (i-1)  do
|  |  p ← p.next
|  |  counter ← counter + 1
|  end
|  if  p ≠ NIL 且 p.next ≠ NIL  then //p 指向第 (i-1)个结点，且待删除结点存在
|  |  deleted_node ← p.next
|  |  p.next ← deleted_node.next
|  |  delete  deleted_node
|  else
|  |  删除位置不合法，退出
|  end
end
```

2.4.2 单链表的基本操作

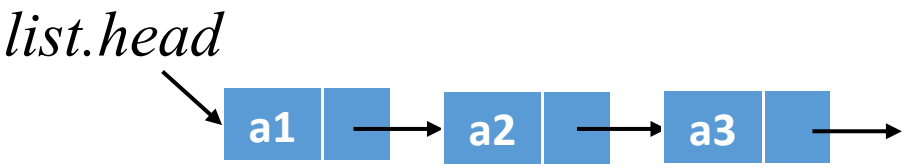
- **如何统一表头及其他位置的插入和删除?**
  - **带头结点的单链表**
    - **头结点不存储数据，真正的元素链接在这个空结点之后**



带头结点的单链表

判断表空:

List.head.next == null



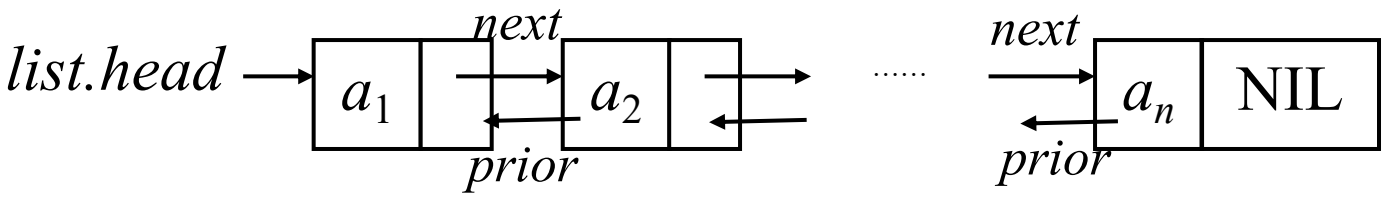
不带头结点的单链表

判断表空:

List.head == null

2.4.3 双向链表

- **单链表存在的问题**
  - 访问给定结点的前一结点需从表头开始遍历
  - 时间复杂度为O(n)
- **双向链表**
  - 在单链表的基础上，每个结点增加一个**指向直接前驱**的指针prior
  - 即每个结点都有两个指针，一个next指向直接后继，另一个prior指向直接前驱。



2.4.3 双向链表的基本操作

- **1. 求表长和查找的操作与单链表相同**
- **2. 插入**
  - **与单链表类似**
  - **指针的更新不同**

代码 2-3 在双向链表中 p 所指结点后面插入 t 所指的结点 DLLInsert(p, t)

输入： 双向链表中某结点指针 p，待插入结点的指针 t

输出： 插入完成后的双向链表 *list*

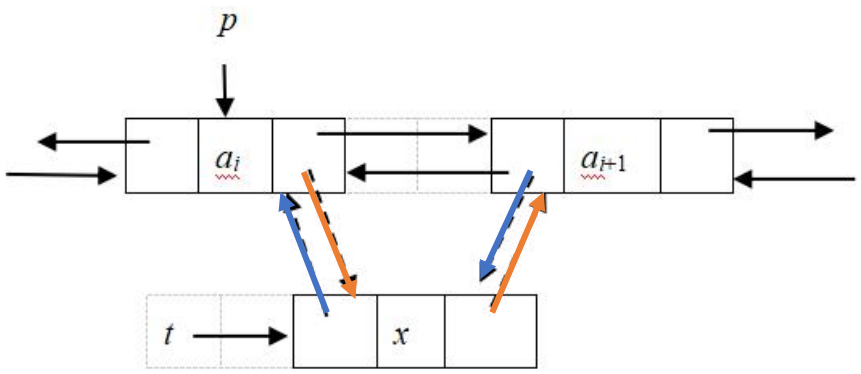
1 $p.next.prior \leftarrow t$

2 $t.next \leftarrow p.next$

3 $p.next \leftarrow t$

4 $t.prior \leftarrow p$

```
t->next = p->next;
t->prior = p;
p->next->prior = t;
p->next = t;
```



2.4.3 双向链表的基本操作

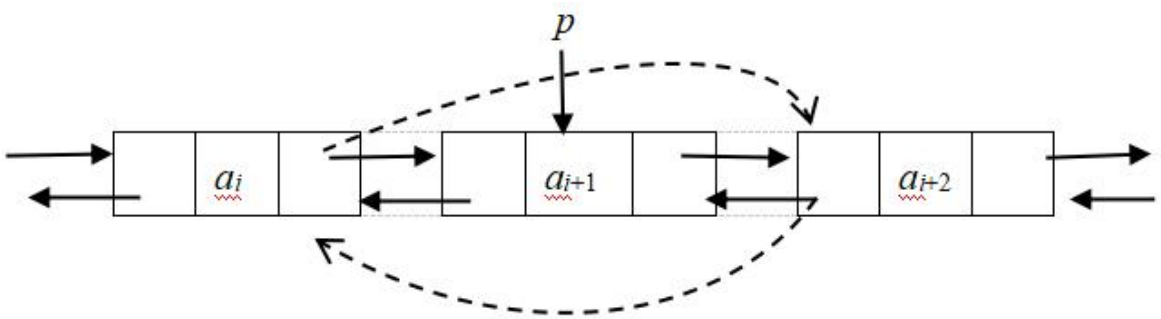
- **3. 删除：删除双向链表中p所指向结点**

代码 2-4 将双向链表中 p 所指的结点删除 DDLDelete(p)

输入： 双向链表中待删除结点指针 p

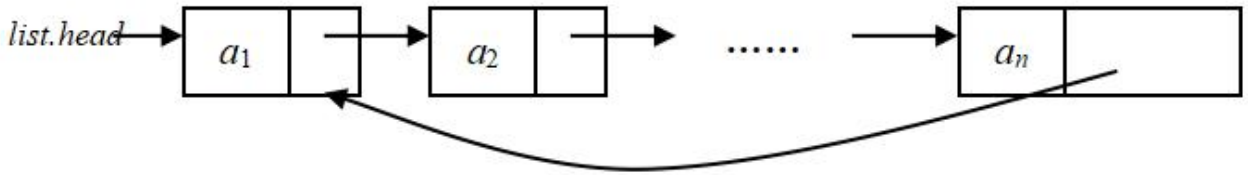
输出： 完成删除后的双向链表 *list*

```
p.next.prior ← p.prior
p.prior.next ← p.next
delete p
```

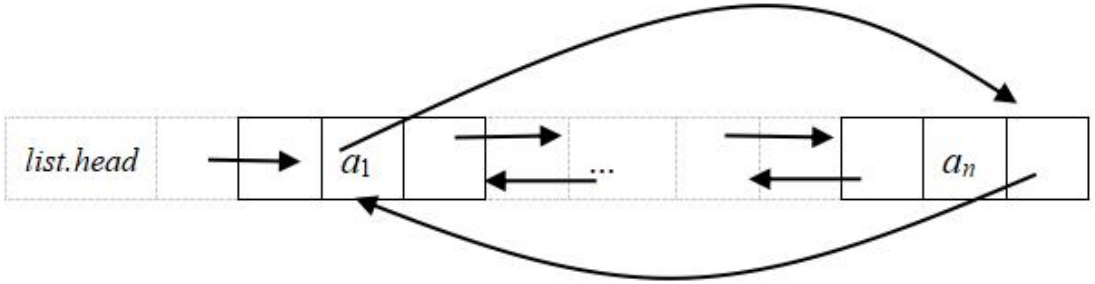


2.4.4 循环链表

- **循环单链表**
  - **链表终端结点的指针指向链表的起始结点**



- **循环双向链表**
  - **让终端结点的后继指针指向链表的起始结点，同时让起始结点的前驱指针指向链表的终端结点**



2.4.4 循环链表

- **循环单链表的遍历**
  - **可以从任意结点start开始遍历整个链表**
  - **遍历结束的条件 p.next != start**

代码 2-5 从单向循环链表中 *start* 指向的结点开始，遍历每个结点 CLLTraverse(*start*)

输入： 起始结点的指针 *start*

输出： Visit 函数处理每个访问结点的结果

```
p ← start //通过指针 p 遍历
do
    Visit(p)
    p ← p.next
while  p ≠ start
```

2.4.5 静态链表

- **静态链表**
  - **用数组存放线性表中的元素，但并不按照元素顺序在数组中依序存放，而是给每个数组元素增加一个域，用于指示线性表中下一个元素的位置（即它在数组中的下标）**
  - **物理存储空间上依赖于数组**
  - **元素逻辑链接关系是采用了链表的思想**
  - **优点：减少了动态开辟内存的开销**

（a,b,c,d,e,f,g,h）

data		g	d	a	e	b	f		h	c	
next	3	8	4	5	6	9	1		0	2	
下标	0	1	2	3	4	5	6	7	8	9	10

2.4.5 静态链表

- **静态链表的遍历**

代码 2-6 将静态链表 *list* 的每个结点访问一遍 SLTraverse(*list*)

输入： 静态链表 *list*

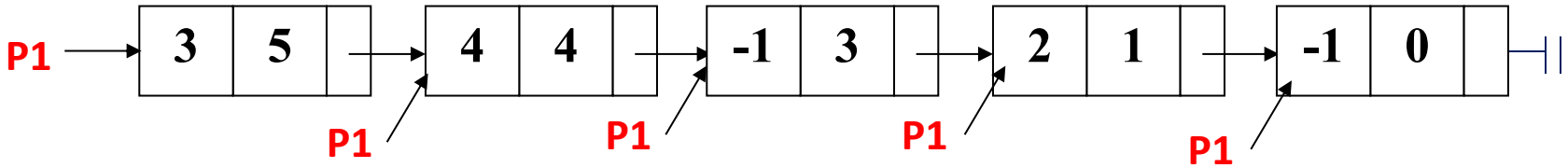
输出： Visit 函数处理每个访问结点的结果

```
p ← list[0].next
while  p ≠ 0  do
    Visit(p)
    p ← list[p].next
end
```

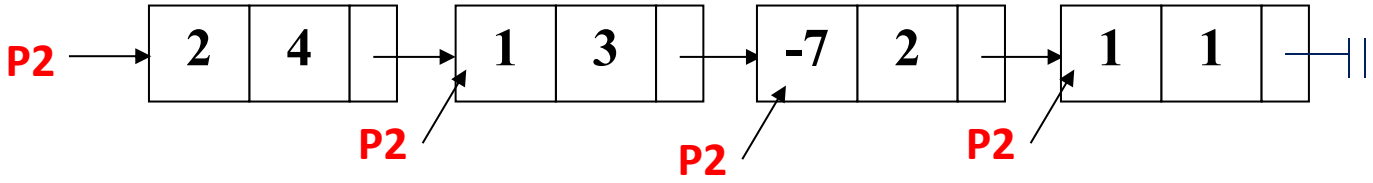
2.5 线性表的应用——一元多项式

- **一元多项式的加法**
  - **p1和p2分别指向两个多项式第一个结点，不断循环比较p1和p2所指的各结点，做不同处理**

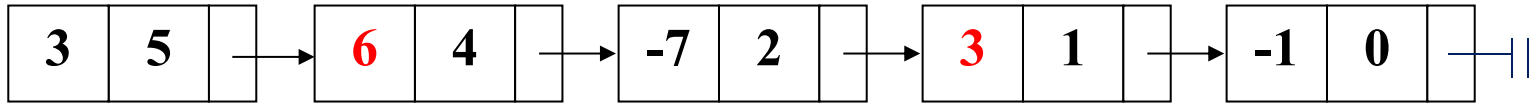
- P1.expon==P2.expon，
  插入求和结点，
  P1=P1.next；
  P2=P2.next



- P1.expon>P2.expon，
  插入P1，P1=P1.next;



- P1.expon<P2.expon,
  插入P2，P2=P.next;



2.5.2 大整数处理

- **大整数表示**
  - 用顺序表中的元素依次表示大整数的个位数、十位数、百位数……
  - n = -23456所对应的digits[] = {6, 5, 4, 3, 2}，length = 5，sign = -1
- **1. 加法运算（假设a+b>0）**
  - （1）首先将这两个大整数的位数对齐（位数较少的大整数的高位补0）
  - （2）将两个大整数对应的位数依次相加或相减，同时处理进位或借位，并将结果存入一个新的大整数*c*中
  - （3）处理加法导致的最高位进位，或减法导致的前导0问题

2.5.2 大整数处理

- **2. 乘法运算**
  - （1）用i和j的二重循环分别枚举大整数a和b的每一位（数组下标从0开始）
  - （2）对于a的第i位数字和b的第j位数字（按从低位到高位的顺序），将它们相乘并**累加至表示计算结果的大整数c的第i+j位**上
  - （3）对于得到的大整数c，从最低位到最高位**依次处理进位**问题，得到最终的计算结果
  - （4）处理符号位：均大于0，则为正；均小于0，则为负

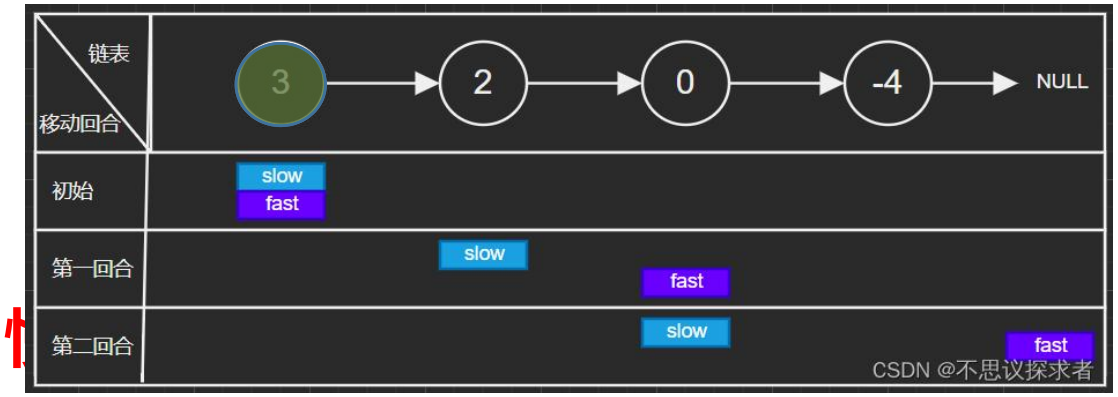
```
      2   3
  x   4   5
  ---------
     10  15
  8  12
  ---------
  8  22  15
```

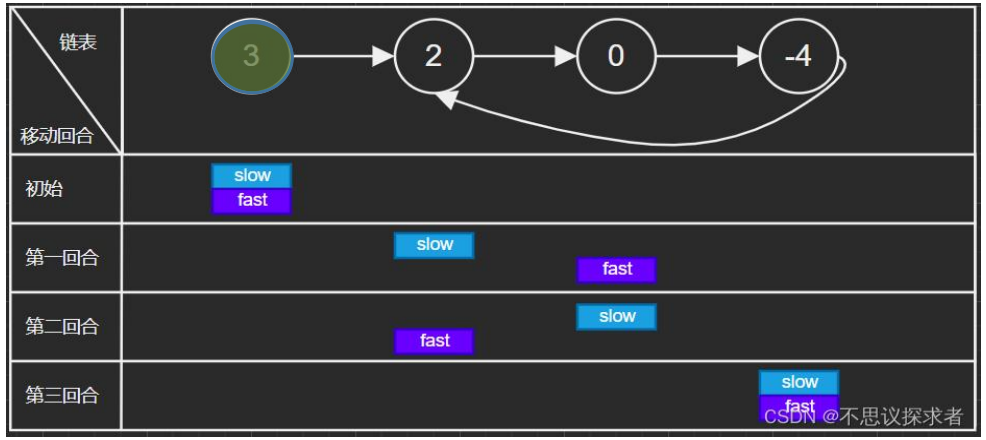
```
  8  22  15
  ---------
  8  23   5
  ---------
 10   3   5
  ---------
1  0   3   5
```

时间复杂度是$O(n^2)$

线性表的应用

- **判定给定的单链表是否存在环?**
  - **Floyd环判断方法**
  - **一个慢指针，一个快指针，初始均指向头节点**
    - **慢指针每次移动一步，快指针每次移动两步**
  - **快指针走到链表尾部，即不存在环**





线性表的应用

- **判定给定的单链表是否存在环**

```
bool LList::hasLoop() {
   Link<Elem> *slowPtr = head, *fastPtr = head;      //init to head
   while (fastPtr->next != nullptr && fastPtr->next->next != nullptr) {
      slowPtr = slowPtr->next;
      fastPtr = fastPtr->next->next;
      if (slowPtr == fastPtr)
         return true;
   }
   return false;
}
```

- **时间复杂度为O(n)，空间复杂度为O(1)**

线性表的应用

- 逆置单链表（带头结点）
  - 循环的将元素插入表头

```
void LList::reverse() {
    if (head->next == nullptr)
        return;                                  //empty list
    tail = head->next;                           // first elment as the tail
    Link<Elem> *temp = head->next->next;         //ignore the first one
    while (temp != nullptr) {
        Link<Elem> *temp_next = temp->next;      //save the next element
        temp->next = head->next;
        head->next = temp;
        temp = temp_next;
    }
    tail->next = nullptr;
}
```

- 时间复杂度为O(n)，空间复杂度为O(1)

练习

- **如何逆置不带头结点的单链表?**

```
void LList::reverse() {
   if (head == nullptr || head->next == nullptr)
      return;                                  //empty list or only has one element
   tail = head;                                // first elment as the tail
   Link<Elem> *temp = head->next;              //ignore the first one
   while (temp != nullptr) {
      Link<Elem> *temp_next = temp->next;      //save the next element
      temp->next = head;
      head = temp;
      temp = temp_next;
   }
   tail->next = nullptr;
}
```

线性表的应用

- **如何找到链表的中间节点？ （只用一次扫描）**
  - 可以使用Floyd环检测算法
  - 快指针到达表尾的时候，慢指针恰好是中间节点

```
Link<E>* LList::middleNode() {
   Link<Elem> *slowPtr = head, *fastPtr = head;      //init to head
   while (fastPtr->next != nullptr && fastPtr->next->next != nullptr) {
      slowPtr = slowPtr->next;
      fastPtr = fastPtr->next->next;
      if (slowPtr == fastPtr)
         return slowPtr;
   }
   return slowPtr;
}
```

  - 时间复杂度为O(n)，空间复杂度为O(1)

线性表的应用

- **有序链表中插入一个结点**

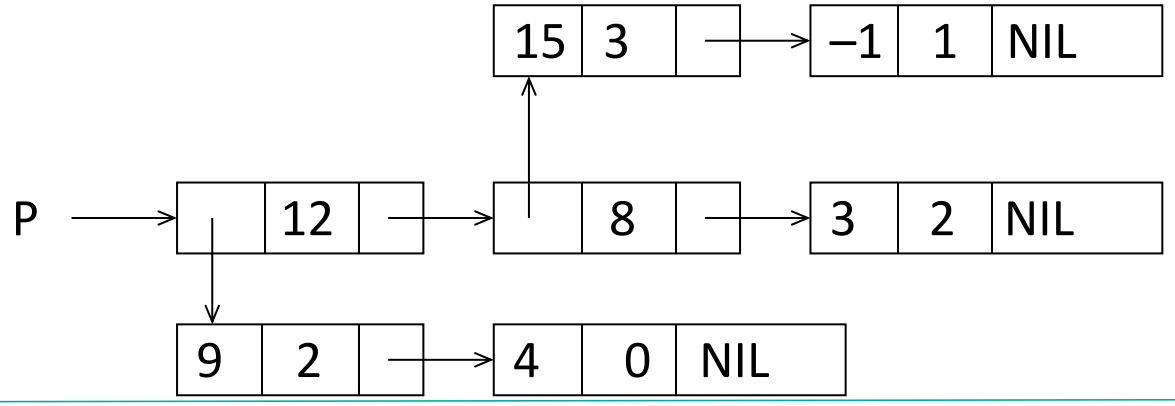
```
void LList::order_insert(const E& it) {
    // find position which larger then it
    Link<Elem> *temp = head;       //ignore the first one
    while (temp->next != nullptr) {
        if (temp->next->element > it) {
            //insert here
            temp->next = new Link<E>(it, temp->next);
            return;
        }
        temp = temp->next;
    }
    //insert at tail
    tail->next = new Link<E>(it, nullptr);
    tail = tail->next;
}
```

- **时间复杂度为O(n)，空间复杂度为O(1)**

2.6 拓展延伸

- **广义表**
  - **广义表是线性表的推广，广义表中的每个元素不仅可以是单元素，也可以是另一个广义表**
    - **线性表中的每个元素均是基本的单元素**
  - **例：二元多项式的表示**

$$P(x,y)=9x^{12}y^2+4x^{12}+15x^8y^3-x^8y+3x^2 \longrightarrow P(x,y)=(9y^2+4)x^{12}+(15y^3-y)x^8+3x^2$$



2.6.2 多维数组和特殊矩阵

- **多维数组的顺序存储**
  - **按行优先顺序存放**：即先存放第0行的元素，再存放第1行的元素，……
  - 例如：二维整形数组 a[3][4]的存放顺序：

a[0][0]	a[0][1]	a[0][2]	a[0][3]	a[1][0]	a[1][1]	a[1][2]	a[1][3]	a[2][0]	a[2][1]	a[2][2]	a[2][3]

  - 二维数组元素a[i][j]的存储位置（地址）

$$l_{ij} = l_{00} + (i \times n_c + j) \times size$$

  - 设n维数组各维大小是（s_1，s_2，……，s_n），第一个元素的地址是$l(0,0,\ldots,0)$，元素占用空间为size个字节，则下标为（i_1, i_2，…, i_n）的元素位置

$$l_{(i1,i2,\ldots in)} = l_{(0,0,\ldots,0)} + (i_1 \times s_2 \times s_3 \times .. \times s_n + i_2 \times s_3 \times \ldots s_n + \ldots i_{n-1} \times s_n + i_n) \times size$$

2.6.2 多维数组和特殊矩阵

- **多维数组的顺序存储**
  - **按行优先顺序存放**
  - **按列优先顺序存放**：即先存放第0列的元素，再存放第1列的元素，……
  - 选用准则
    - 根据计算机访存的**局部性原理**，若应用中经常访问某一行元素，则采用按行优先存放，若经常访问某一列元素，则采用按列优先存放

2.6.2 多维数组和特殊矩阵

- **特殊矩阵：上三角矩阵和下三角矩阵**

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ 0 & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & a_{nn} \end{pmatrix} \qquad \begin{pmatrix} a_{11} & 0 & \cdots & 0 \\ a_{21} & a_{22} & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix}$$

- **压缩空间存储**：只存储上三角获下三角的元素，减少了一半存储空间
- 对于下三角矩阵，设单个元素所占空间为$size$，则a[i][j]的存储位置l_{ij}与矩阵首个元素a[0][0]的地址l_{00} 的关系是：

$$l_{ij} = l_{00} + (i(i+1)/2 + j) \times size，\ i \geq j$$

2.6.3 稀疏矩阵

- **稀疏矩阵**
  - **数值为0的元素数目远多于非0元素的数目，并且非0元素分布没有规律**
- **存储：只存储非0项，以节省存储空间**
  - **三元组的顺序存储**
    - **按行优先（按列优先）顺序存放每个非0项**
    - **每个非0项包括三部分<row, col, value>**

18			2	
	27			
			-4	
23	-1			12

row	0	0	1	2	3	3	3
col	0	3	1	3	0	1	4
value	18	2	27	−4	23	−1	12
	0	1	2	3	4	5	6

  - **三元组表的链式存储**

2.6.3 稀疏矩阵

- **稀疏矩阵的存储**
  - **三元组的顺序存储**
  - **三元组表的链式存储：采用十字链表来存储**
    - **只存储矩阵非0项：结点的数据域包括行坐标row，列坐标col，数值value**
    - **每个结点通过两个指针域，把同行、同列串起来**
      - 行指针(或称为向右指针)Right
      - 列指针（或称为向下指针）Down

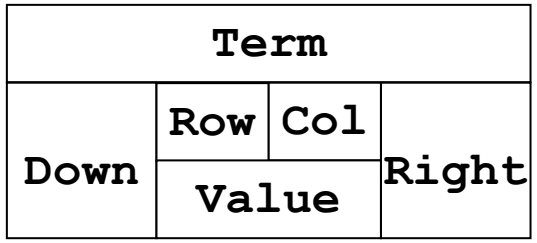
Down	Row	Col	Right
	Value		

    - **每个行链表和列链表都有一个头结点，并采用循环链表的方式存储**
      - 为区分头结点和非0项，增加一个标识域tag，头结点的标识值为head，非0项为term

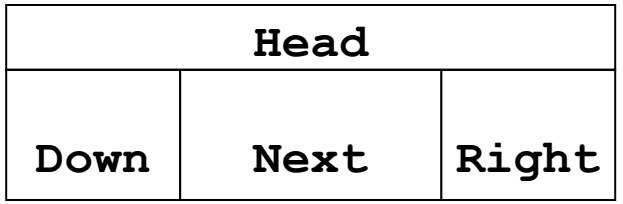
2.6.3 稀疏矩阵

- **稀疏矩阵的十字链表结构**
  - **用一个标识域Tag来区分头结点和非0元素结点**
  - **头节点的标识值为Head，矩阵非0元素结点的标识值为Term**

(a) 结点的总体结构



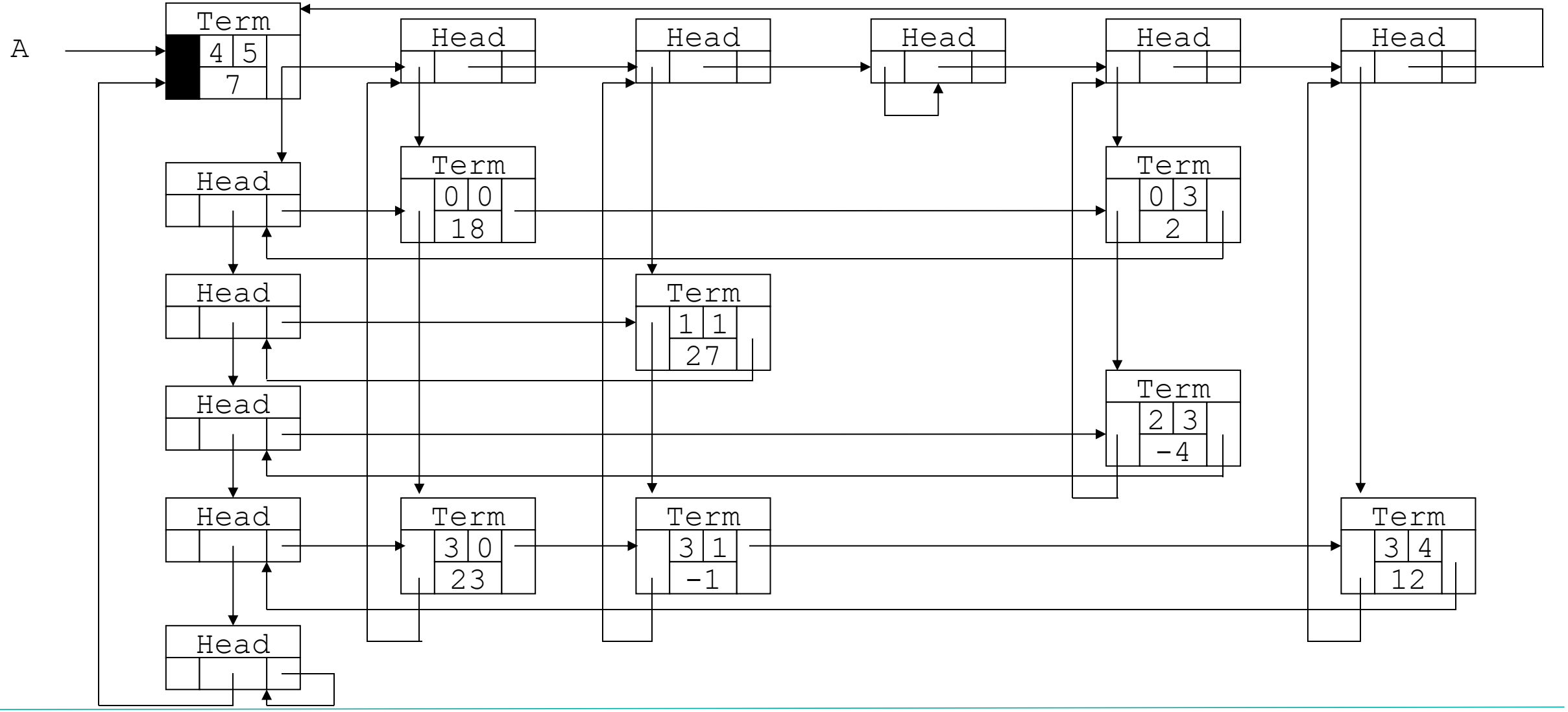
(b) 矩阵非0元素结点



(c) 头结点

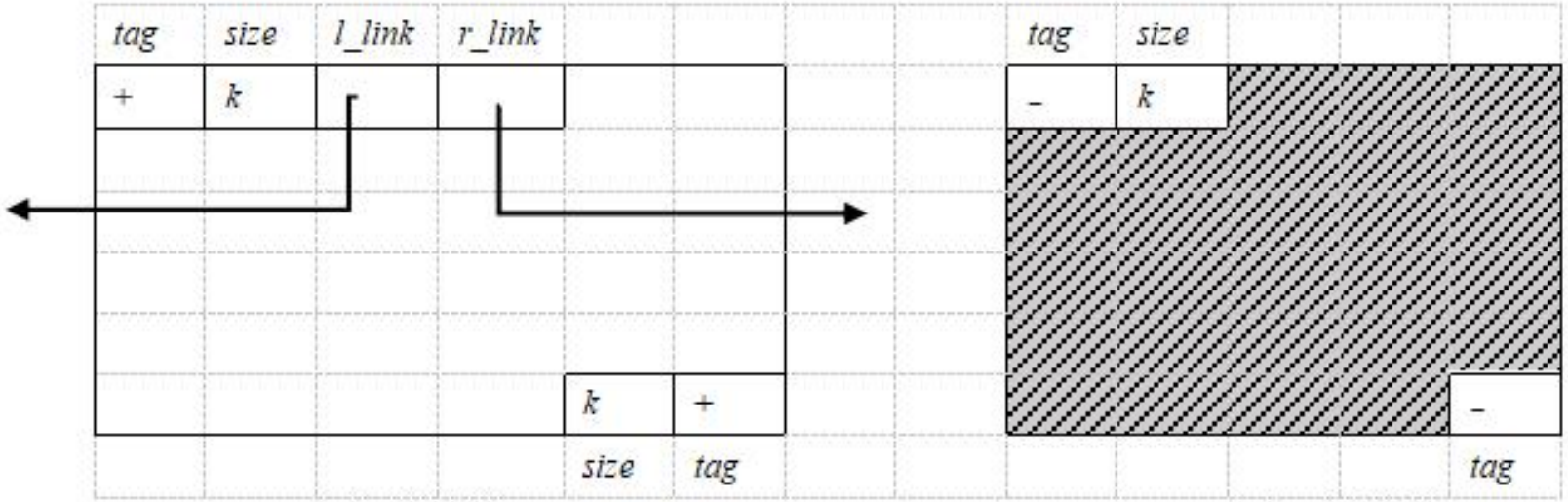
  - **此外，还增加了一个结点用以表示稀疏矩阵的行数、列数、以及非0元素的个数**

示例


A
Term
4 5
7
Head
Head
Head
Head
Head
Head
Term
0 0
18
Term
0 3
2
Head
Term
1 1
27
Head
Term
2 3
-4
Head
Head
Term
3 0
23
Term
3 1
-1
Term
3 4
12
Head

2.7 应用场景：内存管理

- **操作系统空闲内存管理：空闲块的管理，处理申请和回收**
  - **内存空闲块组织成双向链表**
  - **空闲块分配策略：首次适配、最佳适配**
  - **空闲块回收：空闲块合并**



(a) 空闲块结构　　（b）已分配块结构

2.8 小结

- **线性表：**若干数据元素组成的有序序列，其基本操作有插入、删除、查找等
- **线性表实现方法：**顺序存储、链式存储
- **双向链表：**通过设置分别指向后继和前驱结点的两个指针，实现从前向后和从后向前两个方向的访问
- **循环链表：**最后结点的向后指针指向链表的头结点；双向链表头结点的向前指针还指向链表最后结点
- **静态链表：**物理存储依赖于顺序存储结构、逻辑链接关系采用链表思想
- **块状链表：**在传统的链表结构之上采用“分块”的思想进行改进，平衡了查找操作的时间复杂度和插入/删除操作的时间复杂度
- **广义表：**对一般线性表的推广，是一种“表中有表”的数据元素组织方式
- **多维数组：**将元素在空间上按行顺序存储，实现类似一维数组的随机访问
- **特殊矩阵和稀疏矩阵**可以进行压缩存储，减少所需要的存储空间；稀疏矩阵通过十字链表记录非零元素